

Descripción del proyecto

El juego trata de aprender a usar los comandos de Linux de forma divertida para facilitar el aprendizaje

URL del repositorio.

<https://github.com/Msanchez1515/linux-quest-ai->

Uso de la IA — Qué herramientas, modelos o APIs de IA se han utilizado y con qué finalidad. ¿Cómo la has integrado?

*Claude:

para los gráficos

realización de lógica

usar el lenguaje de programación python

Flujos de trabajo — Cómo se ha organizado el proceso de desarrollo (prompting, genético, automatización...)

El desarrollo se organizó de forma interactiva y divertida.

1. Primero se definieron los requisitos del juego (niveles, puntuación y pistas).
- 2.Después se usó prompting con IA (Claude) para generar y corregir el código describiendo en lenguaje natural lo que se necesitaba.
- 3.Cada error encontrado en la terminal se analizaba y se corregía en la siguiente versión, mejorando el juego progresivamente hasta llegar al resultado final.

Explicación del código - Las partes más relevantes, decisiones técnicas y estructura del proyecto.

El proyecto se divide en dos archivos: `app.py`, que gestiona la interfaz y la lógica del juego, y `ai_logic.py`, que contiene los retos, respuestas y pistas por nivel. El estado del juego (puntuación, turnos, fallos) se guarda con `st.session_state`. La decisión más relevante fue sustituir la API de OpenAI por una base de datos local de retos, haciendo el juego gratuito y funcional sin conexión.

Tecnologías utilizadas — Lenguajes, frameworks, servicios, herramientas de IA y otras dependencias.

Python
Streamlit
SVG y CSS
OpenAI API
Claude
Git
Visual Studio Code

Retos y aprendizajes - Dificultades encontradas, cómo se han resuelto y qué se ha aprendido en el proceso.

En mis primeros intentos de desarrollar el juego me encontré con dificultades como que la idea original me parecía muy aburrida, usar (chat gpt) me pedia usar API que se me dificultó de para que no me saliera error y termine cambiando a (claude que no me pedia la clave y no tuve que pagar nada) aprendí a especificarle bien a la IA que necesitaba, que era API, un poquito del lenguaje de python

Limitaciones y mejoras futuras — Qué no se ha podido dar y qué siguientes pasos tendría el proyecto.

Como limitación principal, el juego no puede evaluar comandos con variaciones libres de sintaxis, ya que la corrección se basa en respuestas predefinidas. Además, al eliminar la API de IA, los retos son fijos y no se generan dinámicamente.

Como mejoras futuras se plantea añadir un sistema de registro de usuarios con puntuaciones guardadas, incorporar una terminal simulada donde el jugador ejecute los comandos de verdad, ampliar el banco de retos y, cuando sea viable económicamente, reconectar la IA para generar retos infinitos y personalizados.

Reflexión sobre el uso de la IA — Qué partes de la IA han funcionado bien, cuáles han decepcionado y cómo has calibrado las respuestas.

las que no me agradaron fueron que para la realización de mi juego en chat gpt tenía que pagar para que no me diera error y aprender a usar la clave API

mejore y me agrado la propuesta de claude y la utilice

Instrucciones de instalación y uso — Cómo ejecutar el proyecto desde cero (dependencias, variables de entorno, etc.)

1. Clonar o descargar el proyecto

```
git clone https://github.com/tu-usuario/linux-quest-ai
```

2. Instalar dependencias

```
pip install streamlit
```

3. Ejecutar el juego

```
python -m streamlit run app.py
```

El juego se abrirá automáticamente en el navegador en `http://localhost:8501`. No requiere ninguna clave API ni conexión a internet